Data Science Technical Interview

Round Preparation — Questions & Answers

Python & Programming | Statistics & Probability | Machine Learning | Model Evaluation | SQL | Case Studies

Focused on depth of understanding, not just recall — built for the technical/interview round

Contents

Technical rounds test depth over speed. For each answer, be ready to explain the 'why', not just the definition — interviewers usually follow up with 'can you elaborate?' or 'give me an example from a project'.

1. Python & Programming for Data Science

Core coding fundamentals every data scientist is expected to know fluently.

Q1. What is the difference between a shallow copy and a deep copy in Python?

Answer: A shallow copy creates a new object but inserts references to the same nested objects as the original — so changes to nested/mutable elements affect both copies. A deep copy recursively copies every nested object, creating a fully independent structure.

```
import copy
shallow = copy.copy(original)
deep = copy.deepcopy(original)
```

Why it matters: This matters constantly when copying DataFrames or nested lists/dicts of data — a shallow copy can cause silent, hard-to-debug data corruption.

Q2. How are NumPy arrays different from Python lists, and why are they preferred for data science?

Answer: NumPy arrays store elements of a single fixed data type in contiguous memory, enabling vectorized operations and much faster computation than Python lists, which store references to objects scattered in memory. NumPy also supports broadcasting, element-wise operations, and efficient multi-dimensional indexing that plain lists don't support natively.

Why it matters: Vectorization avoids explicit Python-level loops, which is the main reason NumPy/Pandas code runs orders of magnitude faster than pure Python on large datasets.

Q3. What is the difference between .loc[] and .iloc[] in Pandas?

Answer: .loc[] selects rows and columns using labels (index names or column names), and its slicing is inclusive of the end label. .iloc[] selects using integer positions (0-based), and its slicing is exclusive of the end index, similar to standard Python list slicing.

```
df.loc['a':'c'] # label-based, inclusive
df.iloc[0:3] # position-based, exclusive of 3
```

Why it matters: Mixing up .loc and .iloc is one of the most common real-world Pandas bugs, especially after filtering a DataFrame changes its index.

Q4. What are Python decorators, and why might a data scientist use one?

Answer: A decorator is a function that wraps another function to extend or modify its behavior without changing its source code. In data science, decorators are commonly used to time function execution, cache/memoize expensive computations, or log inputs/outputs during pipeline debugging.

```
from functools import lru_cache

@lru_cache(maxsize=None)
def load_data(path):
return pd.read_csv(path)
```

Why it matters: A very practical use: wrapping a slow data-loading function with @lru_cache to avoid reloading the same data repeatedly.

Q5. How would you handle missing values in a Pandas DataFrame? Name at least three approaches.

Answer: Common approaches include: (1) dropping rows/columns with df.dropna() when missingness is small and random; (2) imputing with mean/median/mode using df.fillna(); (3) using model-based imputation (like KNNImputer or IterativeImputer) for more sophisticated filling; and (4) creating a separate 'is_missing' indicator flag when the missingness itself carries information.

Why it matters: Interviewers want to hear you consider WHY data is missing (MCAR, MAR, MNAR) before picking a method — blindly filling with the mean can distort the distribution.

Q6. What is list comprehension, and how does it compare in performance to a standard for loop?

Answer: List comprehension is a concise syntax for creating a list by applying an expression to each item in an iterable, optionally with a condition. It's generally faster than an equivalent for loop with .append() because the looping happens internally in optimized C code rather than through repeated Python-level append calls.

```
squares = [x**2 for x in range(10) if x % 2 == 0]
```

2. Statistics & Probability

The theoretical backbone interviewers use to test whether you truly understand your models, not just call .fit().

Q7. Explain the bias-variance tradeoff.

Answer: Bias is the error from overly simplistic assumptions in a model, causing it to underfit and miss real patterns. Variance is the error from a model being overly sensitive to small fluctuations in the training data, causing it to overfit. Total expected error can be decomposed as Bias² + Variance + Irreducible Error, so reducing one often increases the other — the goal is finding the sweet spot that minimizes total error on unseen data.

Why it matters: This is arguably the single most-asked conceptual question in data science interviews — practice sketching the classic U-shaped total-error curve while explaining it.

Q8. What is a confidence interval, and how would you interpret a 95% confidence interval for a mean?

Answer: A confidence interval gives a range of plausible values for an unknown population parameter, based on sample data. A 95% confidence interval means that if we repeated the sampling process many times and built an interval each time, about 95% of those intervals would contain the true population mean.

Why it matters: Common trap: it does NOT mean there's a 95% probability the true mean lies in this specific interval — the true mean is fixed, only the interval is random.

Q9. What is A/B testing, and what statistical test would you typically use to compare two groups?

Answer: A/B testing is a controlled experiment comparing two variants (A and B) to see which performs better on a target metric, by randomly assigning users to each group. For comparing conversion rates (proportions), a two-proportion z-test or chi-square test is typically used; for comparing means (like average revenue per user), a two-sample t-test is common.

Why it matters: Always mention checking for statistical significance (p-value) AND practical significance (effect size) — a statistically significant result can still be too small to matter for the business.

Q10. What is the difference between a parametric and a non-parametric statistical test?

Answer: Parametric tests (like the t-test or ANOVA) assume the underlying data follows a specific distribution, usually normal, and rely on estimating parameters like mean and variance. Non-parametric tests (like the Mann-Whitney U test or Kruskal-Wallis test) make no such distributional assumption and instead work with ranks, making them more robust when normality doesn't hold or sample sizes are small.

Why it matters: Mention: non-parametric tests are generally less statistically powerful when the parametric assumptions actually do hold, which is the tradeoff for their flexibility.

Q11. What is regularization, and how do L1 and L2 regularization differ?

Answer: Regularization adds a penalty term to a model's loss function to discourage overly complex models and reduce overfitting. L1 regularization (Lasso) adds the sum of absolute values of coefficients, which can shrink some coefficients exactly to zero — performing implicit feature selection. L2 regularization (Ridge) adds the sum of squared coefficients, which shrinks coefficients smoothly toward zero but rarely to exactly zero.

Why it matters: If asked which to use: L1 when you suspect many features are irrelevant (want sparsity); L2 when most features are likely useful but need to control their magnitude.

Q12. What is the difference between covariance and correlation?

Answer: Covariance measures the direction of the linear relationship between two variables, but its magnitude is not standardized and depends on the units of the variables, making it hard to compare across datasets. Correlation (like Pearson's r) is covariance normalized by the product of the standard deviations of both variables, producing a unitless value between -1 and 1 that's directly comparable across different variable pairs.

Why it matters: Formula to keep handy: Correlation = Covariance(X,Y) / (StdDev(X) × StdDev(Y)).

3. Machine Learning Algorithms & Concepts

The heart of the technical round — expect deep follow-ups on 'why' and 'when', not just 'what'.

Q13. Explain how a Decision Tree decides where to split, and name two common splitting criteria.

Answer: A Decision Tree evaluates every possible split point on every feature and picks the one that produces the greatest improvement in 'purity' of the resulting child nodes. Two common criteria are Gini Impurity (measures the probability of misclassifying a randomly chosen element) and Entropy/Information Gain (measures disorder, borrowed from information theory). Both aim to create child nodes that are as homogeneous as possible.

Why it matters: Follow-up to expect: how does a tree overfit, and how does pruning or setting max_depth help?

Q14. How does Random Forest reduce overfitting compared to a single Decision Tree?

Answer: Random Forest builds many decision trees, each trained on a bootstrapped (random, with-replacement) sample of the data and considering only a random subset of features at each split. This introduces diversity among the trees, so their individual errors tend to cancel out when their predictions are averaged (regression) or voted (classification) — reducing the variance that a single deep tree would have.

Why it matters: Key term to mention: 'bagging' (Bootstrap Aggregating) is the general technique Random Forest is built on.

Q15. What is the difference between Bagging and Boosting?

Answer: Bagging trains multiple models independently and in parallel on random subsets of data, then combines them by averaging/voting to reduce variance (e.g. Random Forest). Boosting trains models sequentially, where each new model focuses on correcting the errors made by previous models, combining them in a weighted way to reduce bias (e.g. AdaBoost, Gradient Boosting, XGBoost).

Why it matters: Quick line: 'Bagging reduces variance by parallel averaging; Boosting reduces bias by sequential correction.'

Q16. Why does Gradient Descent sometimes fail to converge, and how can the learning rate cause this?

Answer: If the learning rate is too high, updates overshoot the minimum and the loss can oscillate or diverge instead of settling down. If the learning rate is too low, convergence becomes extremely slow and the model may get stuck in a shallow local minimum or plateau before reaching a good solution. Techniques like learning rate schedules, momentum, or adaptive optimizers (Adam, RMSprop) help address this.

Why it matters: A good sketch to describe verbally: too high = zig-zagging past the valley; too low = crawling toward it very slowly.

Q17. Explain the intuition behind Support Vector Machines (SVM) and what a 'kernel' does.

Answer: An SVM finds the hyperplane that separates classes with the maximum margin — the largest possible distance to the nearest data points of either class (the 'support vectors'). When data isn't linearly separable in its original space, a kernel function (like RBF or polynomial) implicitly maps the data into a higher-dimensional space where a linear separator can be found, without explicitly computing that transformation (the 'kernel trick').

Why it matters: Mention the 'kernel trick' by name — it's the specific term interviewers listen for.

Q18. What is the vanishing gradient problem in deep neural networks, and how is it typically addressed?

Answer: In deep networks trained via backpropagation, gradients can shrink exponentially as they are propagated backward through many layers, especially with saturating activation functions like sigmoid or tanh — causing earlier layers to learn extremely slowly or stop learning altogether. It's commonly addressed using ReLU-family activation functions (which don't saturate for positive inputs), batch normalization, residual/skip connections (as in ResNet), and careful weight initialization (like He or Xavier initialization).

Why it matters: If discussing RNNs specifically, mention LSTM/GRU gates as the architecture-level fix for vanishing gradients over long sequences.

Q19. What is the difference between K-Means clustering and Hierarchical clustering?

Answer: K-Means requires you to specify the number of clusters (k) in advance, and iteratively assigns points to the nearest centroid before recalculating centroids, converging quickly even on large datasets. Hierarchical clustering builds a tree of clusters (a dendrogram) either by merging (agglomerative) or splitting (divisive) clusters step by step, and does not require choosing k upfront, but is computationally more expensive for large datasets.

Why it matters: Mention that you can cut a dendrogram at different heights to explore different numbers of clusters after the fact — a flexibility K-Means doesn't offer.

Q20. How would you handle a severe class imbalance problem, for example fraud detection where only 1% of transactions are fraudulent?

Answer: Approaches include: resampling techniques like SMOTE (synthetic oversampling of the minority class) or random undersampling of the majority class; using class-weighted loss functions so the model penalizes misclassifying the minority class more heavily; choosing appropriate evaluation metrics (Precision, Recall, F1, PR-AUC) instead of plain accuracy, since accuracy is misleading here; and considering anomaly-detection framing instead of standard classification.

Why it matters: Always mention that accuracy is a trap in imbalanced problems — a model predicting 'not fraud' every time would already be 99% accurate but useless.

4. Model Evaluation & Validation

Knowing HOW to judge a model correctly is as important to interviewers as building the model itself.

Q21. What is k-fold cross-validation, and why is it preferred over a single train-test split?

Answer: K-fold cross-validation splits the data into k equal parts (folds); the model is trained on k–1 folds and validated on the remaining fold, repeating this k times so every fold is used for validation exactly once, then averaging the results. It's preferred over a single train-test split because it uses all the data for both training and validation across iterations, giving a more reliable and less variance-prone estimate of model performance.

Why it matters: Common value to mention: k=5 or k=10 is standard; higher k means less bias but more compute time and slightly higher variance in the estimate.

Q22. What is the ROC curve and what does AUC represent?

Answer: The ROC (Receiver Operating Characteristic) curve plots the True Positive Rate against the False Positive Rate at various classification thresholds. AUC (Area Under the Curve) summarizes this into a single number between 0 and 1, representing the probability that the model ranks a randomly chosen positive example higher than a randomly chosen negative example — an AUC of 0.5 means no better than random guessing, while 1.0 is a perfect ranking.

Why it matters: Mention that ROC-AUC can be misleadingly optimistic on highly imbalanced datasets — Precision-Recall AUC is often more informative there.

Q23. What is the F1 score, and when would you prioritize it over accuracy?

Answer: The F1 score is the harmonic mean of Precision and Recall: F1 = 2 × (Precision × Recall) / (Precision + Recall). It's prioritized over accuracy whenever classes are imbalanced or when both false positives and false negatives carry meaningful cost — for example, in medical diagnosis or fraud detection, where accuracy alone can hide poor performance on the minority (and usually more important) class.

Why it matters: The harmonic mean punishes extreme imbalance between precision and recall more than a simple average would — worth mentioning if asked 'why harmonic mean, not just average?'

Q24. What is data leakage, and can you give an example of how it might occur?

Answer: Data leakage happens when information from outside the legitimate training data — often information that wouldn't be available at real prediction time — accidentally influences model training, leading to unrealistically good performance during evaluation that doesn't hold up in production. Example: scaling/normalizing the entire dataset (including the test set) before splitting into train/test, so the test set's statistics leak into the training process; or including a feature that's only known after the outcome occurs.

Why it matters: The safe pattern to state: fit any preprocessing (scalers, encoders, imputers) ONLY on the training set, then transform the test set using those already-fitted parameters.

Q25. What is hyperparameter tuning, and what's the difference between Grid Search and Random Search?

Answer: Hyperparameter tuning is the process of systematically searching for the combination of model settings (like learning rate, tree depth, or regularization strength) that yields the best validation performance. Grid Search exhaustively tries every combination in a predefined grid of values, guaranteeing the best combination within that grid but scaling poorly as the number of hyperparameters grows. Random Search samples random combinations from specified distributions, which is often more efficient at finding good hyperparameters in high-dimensional spaces because it doesn't waste effort on unimportant parameter combinations.

Why it matters: For advanced discussion, mention Bayesian Optimization as a smarter alternative that uses past results to decide which combination to try next.

5. SQL for Data Science

Practical query-writing ability — often tested live on a shared screen or coding platform.

Q26. Write a query to find the top 3 highest-paid employees in each department from a table Employees(id, name, department, salary).

Answer: Using a window function to rank salaries within each department, then filtering to the top 3:

```
SELECT * FROM (
SELECT *,
DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk
FROM Employees
) ranked
WHERE rnk <= 3;
```

Why it matters: DENSE_RANK() is preferred over ROW_NUMBER() here since it correctly handles tied salaries within the same department instead of arbitrarily breaking the tie.

Q27. How would you find customers who placed orders in every month of 2025 from an Orders(customer_id, order_date) table?

Answer: Group by customer, count distinct months, and filter for exactly 12 distinct months:

```
SELECT customer_id
FROM Orders
WHERE YEAR(order_date) = 2025
GROUP BY customer_id
HAVING COUNT(DISTINCT MONTH(order_date)) = 12;
```

Why it matters: This pattern — group, count distinct, then HAVING = target count — generalizes to many 'present in every period' style business questions.

Q28. What is a Common Table Expression (CTE), and why might you use one instead of a subquery?

Answer: A CTE (defined using WITH ... AS) is a named, temporary result set that exists only for the duration of a single query, which can be referenced one or more times later in that query. CTEs are preferred over nested subqueries mainly for readability and maintainability — especially for multi-step logic or when the same intermediate result needs to be reused multiple times, avoiding rewriting (or recomputing) the same subquery repeatedly.

```
WITH dept_avg AS (
SELECT department, AVG(salary) AS avg_sal
FROM Employees GROUP BY department
)
SELECT e.name, e.salary, d.avg_sal
FROM Employees e JOIN dept_avg d ON e.department = d.department
WHERE e.salary > d.avg_sal;
```

Q29. What is the difference between a clustered and a non-clustered index?

Answer: A clustered index determines the physical order in which rows are stored on disk, so a table can have only one clustered index. A non-clustered index is a separate structure that stores pointers back to the actual data rows, so a table can have several non-clustered indexes to speed up different types of queries.

Why it matters: Interviewers may follow up with: 'why not add indexes to every column?' — Answer: indexes speed up reads but slow down writes (INSERT/UPDATE/DELETE) and consume extra storage.

Q30. How would you calculate a 7-day moving average of daily sales using SQL?

Answer: Use a window function with a frame that spans the current row and the 6 preceding rows:

```
SELECT sale_date, sales,
AVG(sales) OVER (
ORDER BY sale_date
ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
) AS moving_avg_7d
FROM DailySales;
```

Why it matters: This ROWS BETWEEN ... PRECEDING AND CURRENT ROW pattern is the standard way to compute rolling metrics directly in SQL without exporting to Python/Pandas.

6. Scenario & Case-Study Questions

Open-ended, judgment-based questions that test how you'd actually approach a real business problem.

Q31. Your regression model has high training accuracy but performs poorly on the test set. Walk through how you would diagnose and fix this.

Answer: This is a classic overfitting symptom. Steps to diagnose: (1) compare training vs validation/test error to confirm the gap; (2) check if the model is too complex relative to the amount of data (e.g. too many features, too deep a tree); (3) check for data leakage that may be inflating training performance unrealistically. Fixes include: adding regularization (L1/L2), reducing model complexity, gathering more training data, using cross-validation for more reliable tuning, and applying dropout/early stopping if using neural networks.

Why it matters: Structure your answer as Diagnose → Confirm → Fix — interviewers reward a clear problem-solving process over jumping straight to a solution.

Q32. A stakeholder asks you to build a model to predict customer churn. What are the first few questions you would ask before starting?

Answer: Key clarifying questions include: How is 'churn' precisely defined (e.g. no purchase in 90 days, or explicit cancellation)? What business action will be taken based on the prediction (this affects whether precision or recall matters more)? What historical data is available, and over what time period? Is there a need for the model to be interpretable (e.g. for a business team to act on specific reasons), or is raw predictive performance the main priority?

Why it matters: This tests whether you jump straight into modeling or first understand the business problem — interviewers specifically look for the latter.

Q33. How would you explain a complex model's prediction (say, a Random Forest or XGBoost model) to a non-technical stakeholder?

Answer: Use feature importance or SHAP (SHapley Additive exPlanations) values to identify which factors most influenced a specific prediction, then translate that into plain business language — e.g. 'this customer was flagged as likely to churn mainly because their usage dropped 40% in the last month and they contacted support twice recently.' Avoid technical jargon like 'Gini impurity' or 'gradient boosting iterations' in the explanation itself.

Why it matters: Mentioning SHAP or LIME by name signals awareness of model interpretability tools, which is increasingly expected in industry roles.

Q34. You're given a dataset where 30% of values in an important numeric column are missing. How would you decide on an approach, and what would you avoid doing blindly?

Answer: First, investigate WHY the data is missing — check if missingness correlates with other variables or the target (MAR/MNAR) versus being random (MCAR), since this determines whether simple imputation is safe. With 30% missing, dropping rows outright would lose too much data, so options include model-based imputation (KNN or iterative imputation), or adding a 'was_missing' indicator flag alongside a reasonable imputed value so the model can still use the signal that data was missing. Avoid blindly filling every missing value with the mean without first checking if that distorts the distribution or hides a meaningful pattern.

Why it matters: This question tests statistical judgment, not just knowledge of the fillna() function — always mention investigating the missingness mechanism first.

How to Prepare Using This Guide

- Practice saying each answer out loud, not just reading it — technical interviews are verbal, and fluency matters.
- For every ML concept, be ready with a concrete example from a personal project or coursework where you applied it.
- Actually run the SQL and Python code snippets yourself in an editor so you can adapt them live if asked to modify one.
- For the scenario questions, practice structuring your answer as Clarify → Diagnose → Approach → Fix, rather than jumping straight to a solution.
- Expect interviewers to ask 'why' after almost every answer — prepare a one-line justification for each technique you mention.

Best of luck with your technical interview round!